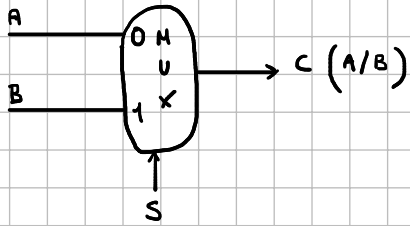


COOSA È UN MULTIPLEXER? A CHE COSA SERVE? DOVE LO POSSIAMO TROVARE?

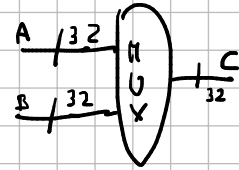
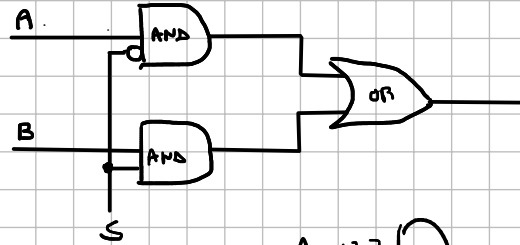
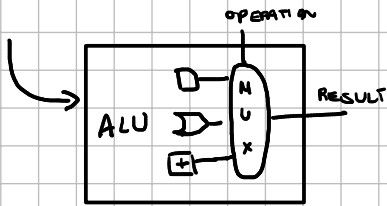
IL **MULTIPLEXER** È UN SELETTORE DIGITALE, LA CUI FUNZIONE È QUELLA DI SCEGLIERE/SELEZIONARE UN INPUT (DATO) CON UN SEGNALE DI CONTROLLO (S).



QUINDI CON $S=0$ (BASSO) IL MULTIPLEXER "LASCIA PASSARE" (SELEZIONA) IL DATO A, INVECE CON $S=1$ LASCIA PASSARE B.

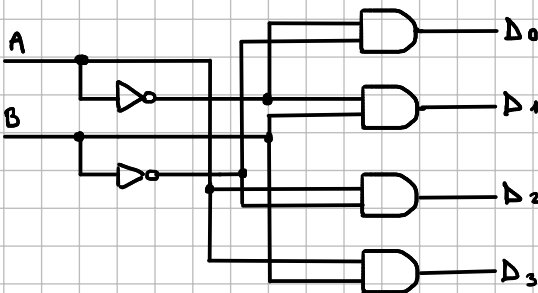
IL SUO CIRCUITO PUÒ ESSERE REALIZZATO NEL SEGUENTE MODO:

LO POSSIAMO TROVARE IN MOLTI CONTESTI COME NELLA ALU PER SELEZIONE L'OPERAZIONE DA ESEGUIRE (O MEGLIO LASCIAR PASSARE), DOVE S SI CHIAMA "OPERATION".



COOSA È UN DECODER? A CHE COSA SERVE? DOVE LO POSSIAMO TROVARE?

UN **DECODER** RICEVE IN INGRESSO UN NUMERO BINARIO n , E ATTIVA (PORTA A VALORE LOGICO DI 1) ESCLUSIVAMENTE UNA DELLE 2^n USCITE.



LO POSSO TROVARE NEI CIRCUITI DI **SRAM** E **DRAM** PER SELEZIONARE LA RIGA NELLA STRUTTURA A MATRICE, OPPURE NEL **REGISTER FILE** PER SELEZIONARE LE CELLE DI MEMORIA IN CUI ANDRANNO MEMORIZZATI DEI DATI.

SE METTO $A=0$ E $B=0$ PORTO A LIVELLO LOGICO 1 D_0 .

A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

CHE COSA È UN FULL ADDER? COME SI FANNO LE SOTTRAZIONI?

UN **FULL ADDER** È UN CIRCUITO SOMMATORE "COMPLETO", CAPACE DI SOMMARE 2 BIT ($A+B$) ED UN EVENTUALE **CARRY-IN** (RIPORTO) IN INPUT E DI GENERARE UNA **SOMMA** E UN **CARRY-OUT** COME OUTPUT.

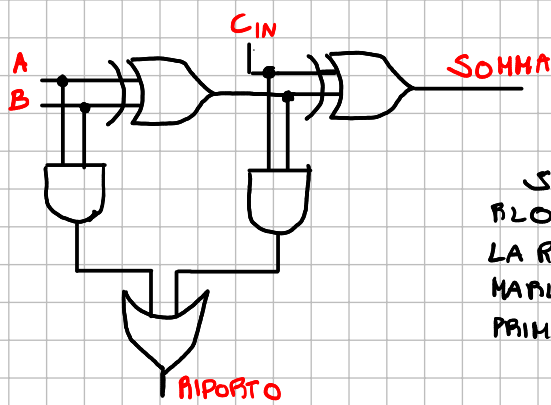
A	B	C_{IN}	SOMMA	C_{OUT}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned}
 \text{SOMMA} &= \bar{A}\bar{B}C_{IN} + \bar{A}B\bar{C}_{IN} + A\bar{B}\bar{C}_{IN} + ABC_{IN} = \\
 &= C_{IN}(\bar{A}\bar{B} + AB) + \bar{C}_{IN}(\bar{A}B + A\bar{B}) = C_{IN}(\overline{A \oplus B}) + \\
 &\quad \bar{C}_{IN}(A \oplus B) = \boxed{A \oplus B \oplus C}
 \end{aligned}$$

$$\begin{aligned}
 C_{OUT} &= \bar{A}BC_{IN} + \bar{A}B\bar{C}_{IN} + A\bar{B}\bar{C}_{IN} + ABC_{IN} = \\
 &= C_{IN}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{IN} + C_{IN}) = \boxed{C_{IN}(A \oplus B) + AB}
 \end{aligned}$$

IL SUO CIRCUITO:

PER COSTRUIRE UN SOMMATORE A N BIT CI VOGLIAMO N SOMMATORI COMPLETI.



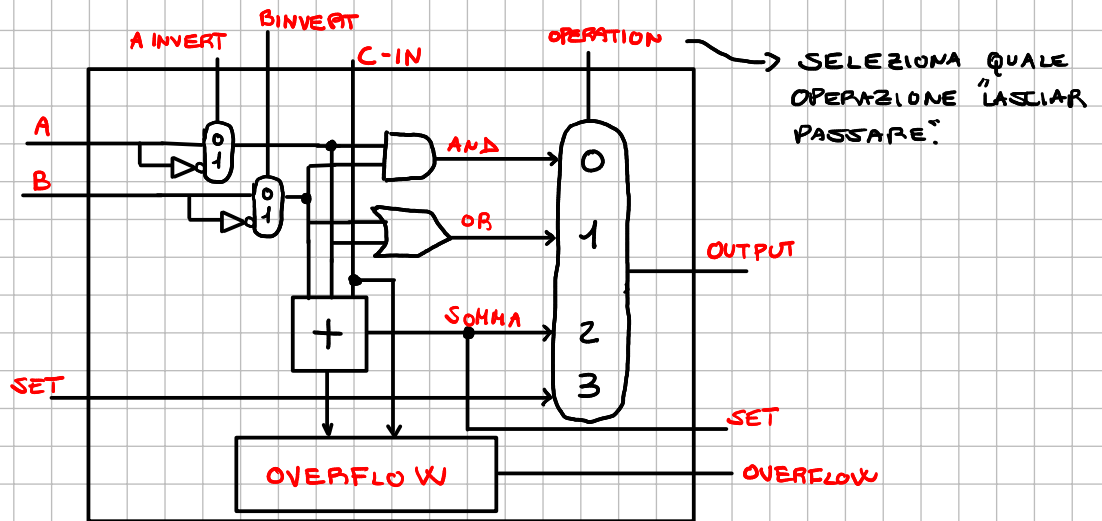
PER FARE LE SOTTRAZIONI SI AGGIUNGE UN INVERTITORE CONTROLLATO SU UNO DEI DUE INPUT PER CONVERTIRLO COMPLEMENTO A 1, E SEGUENDO LA RELAZIONE $\bar{N} = N + 1$, POSSO TRASFORMARLO IN C2 METTENDO IL C-IN DEL PRIMO SOMMATORE A VALORE LOGICO 1.

COSE È UNA ALU? COME È FORMATA? COME È STATO IMPLEMENTATO IL NOR, L'SLT, L'OVERFLOW E ZERO?

UNA ALU (ARITHMETIC LOGIC UNIT) È IL COMPONENTE DELLA CPU CHE SVOLGE LE OPERAZIONI ARITMETICHE E LOGICHE COME SOMMA, AND, OR, NOR E SLT (SET LESS THAN) PER CONFRONTARE GLI INPUT CHE ARRIVANO DAI REGISTRI.

COME È FATTA?

ALU A 1 BIT



IL NOR È STATO IMPLEMENTATO APPLICANDO LA LEGGE DI DE MORGAN OVVERO:

$$A \cdot B = \overline{\overline{A} + \overline{B}} \rightarrow \text{NOR}$$

$$\neg(A+B) = \neg A \cdot \neg B = \bar{A} \cdot \bar{B}$$

QUINDI APPLICANDO UN INVERTITORE CONTROLLATO (CON SEGNALE DI CONTROLLO A-INVERT E B-INVERT) SU ENTRAMBI GLI INPUT E USANDO LA AND GIÀ PRESENTE.

IL FULL ADDER PRENDE IN INPUT A, B (O A E $\bar{B}$) E C-IN (CARRY-IN, NELLA PRIMA ALU VIENE MESSO A 1 PER IL C2) CHE GENERA LA SOMMA E IL CARRY-OUT, FONDAMENTALE PER IL CALCOLO DELL'OVERFLOW NELL'ULTIMA ALU (PIÙ SIGNIFICATIVA).

IL CALCOLO DELL'OVERFLOW AVVIENE CONFRONTANDO IL CARRY-IN E CARRY-OUT DELLA ALU PIÙ SIGNIFICATIVA, SE SONO DIVERSI SI SEGNA LA OVERFLOW, QUESTA LOGICA VIENE IMPLEMENTATA CON UNO XOR.

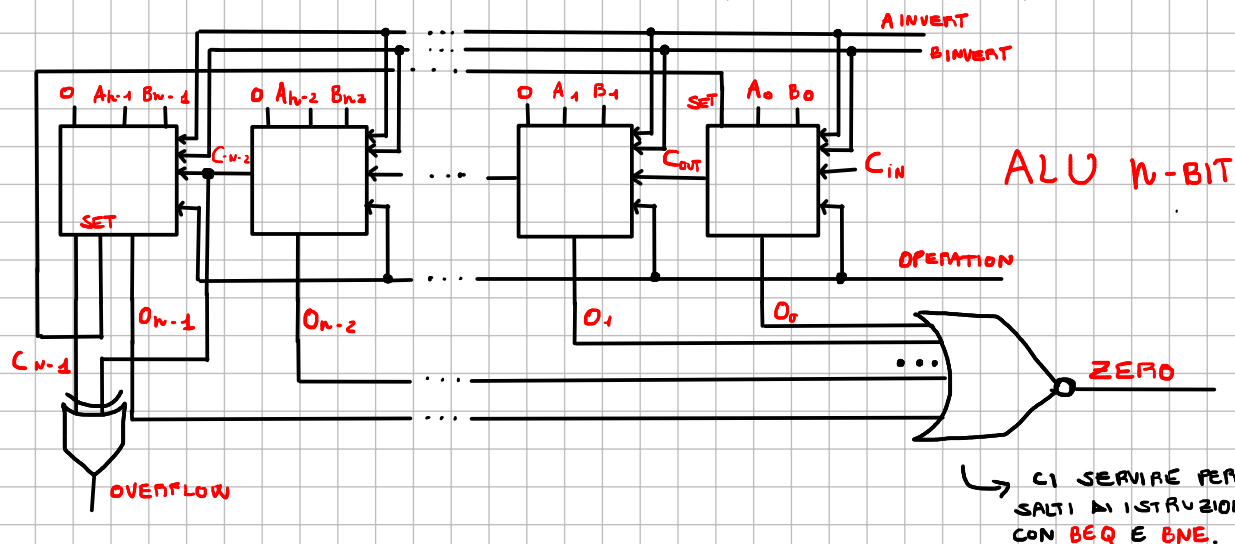
OGNI CPU POSSIENE L'INPUT SET CHE PER TUTTE LE ALU È INIZIALMENTE IMPOSTATO A 0, SERVE PER IL CALCOLO DELL'SLT (SET LESS THAN) OVVERO L'ASSEGNAZIONE DI UN VALORE BOOLEANO (0 = FALSO, 1 = VERO) IN SEGUITO AL CONFRONTO TRA IL CONTENUTO DI 2 REGISTRI.

UN ESEMPIO DEL COMANDO SLT È:

$$\text{SLT } X_3, X_{11}, X_{20} \quad \text{OVVERO: } \text{IF } (X_{11} < X_{20}) \text{ THEN } X_3 = 1 \text{ ELSE } X_3 = 0$$

QUINDI PER CONTROLLARE CHE $X_{11} < X_{20} \Leftrightarrow X_{11} - X_{20} < 0$, BISOGNA CONTROLLARE CHE LA LORO SOTTRAZIONE SIA NEGATIVA, OVVERO L'ALU PIÙ SIGNIFICATIVA (QUELLA CHE GENERA IL SEGNO), DEVE INVIARE LA PROPRIA SOMMA (SEGNO) ALLA ALU MENO SIGNIFICATIVA, COLLEGANDO FISICAMENTE L'OUTPUT CHIAMATO ANCH'ESSO SET DELLA ALU PIÙ SIGNIFICATIVA VERSO L'INPUT SET DELLA ALU MENO SIGNIFICATIVA. IN MODO TALE CHE QUANDO VERRÀ SELEZIONATA DAL MUX CON OPERATION IL RISULTATO SIA UNA SERIE DI ZERO E UN 1 (IN CASO LA CONDIZIONE SIA VERA) ALLA CIFRA MENO SIGNIFICATIVA CHE VERRÀ MEMORIZZATA DA X_3 IN QUESTO CASO.

INFINE PER CAPIRE SE 2 NUMERI SONO UGUALI LA LORO SOTTRAZIONE DEVE DARE ZERO, QUINDI VENGONO COLLEGATI TUTTI GLI **OUTPUT SOMMA** DELLE ALU IN UN UNICO **NOR**, CHE DA OUTPUT 1 QUANDO TUTTI GLI INPUT SONO A ZERO:

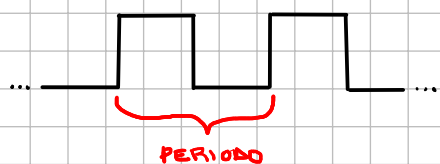


CO SA È IL CLOCK? A COSA SERVE? DOVE VIENE USATO?

IL **CLOCK** È UN COMPONENTE DELLA CPU CHE GENERA IMPULSI, OVERO SEGNALI CHE OSCILLANO TRA 0 E 1, CON PERIODO FISSO. DURANTE IL PERIODO I SEGNALI PASSANO DA BASSA TENSIONE (0) AD ALTA TENSIONE (1) E VICEVERSA, SI DEFINISCONO COSÌ I **FRONTI DI SALITA** E I **FRONTI DI DISCESA**, MOMENTI IN CUI POSSONO AVVENIRE **CAMBIAMENTI DI STATO** (CON CIRCUITI SEQUENZIALI).

IL CLOCK SERVE PER **ORDINARE GLI EVENTI**, E IL SUO PERIODO DEVE ESSERE ABBASTANZA LUNGO PER PROPAGARE GLI INPUT DALLA MEMORIA FINO AI **CIRCUITI COMBINATORI**, FINO A STABILIZZARSI IN OUTPUT.

PERCHÉ LENTI!



→ RICORDIAMO

$$f = \frac{1}{\text{PERIODO}}$$

$$T_{\text{CPU}} = \frac{I \times \text{CPI}}{f}$$

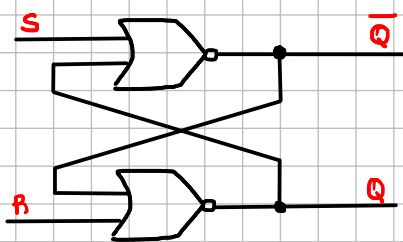
→ + È ALTA, + È VELOCE LA CPU.

IL CLOCK VIENE USATO PER ELIMINARE IL PROBLEMA DELLA **TRASPARENZA** NEI CIRCUITI COME IL **LATCH-D**, PERMETTENDO DI CREARE MEMORIE COME IL **FLIP-FLOP D** CHE SI "ATTIVA" DURANTE I FRONTI DI SALITA O DISCESA DEL CLOCK.

CO SA IL LATCH SET-RESET? IL LATCH-D? IL FLIP FLOP D?

IL **LATCH SET-RESET** È UNA MEMORIA A 1 BIT, DOTATA DI 2 SEGNALI IN INPUT OVERO **S** (SET) E **R** (RESET) CHE PERMETTONO RISPETTIVAMENTE DI SCRIVERE 1 (SET $Q=1$) E 0 (RESET) NELL'USCITA **Q** E RISPETTIVAMENTE IL SUO NEGATO $\bar{Q}$, UTILIZZANDO PORTE **NOR**.

IL SUO CIRCUITO:



A	B	A non B
0	0	1
0	1	0
1	0	0
1	1	0

CASI:

- $S=0, R=0$: MANTIENI IL VALORE PRECEDENTE.

- $S=1, R=0$: SCRIVI 1 SU Q ($\bar{Q}=0$)

- $S=0, R=1$: SCRIVI 0 SU Q ($\bar{Q}=1$)

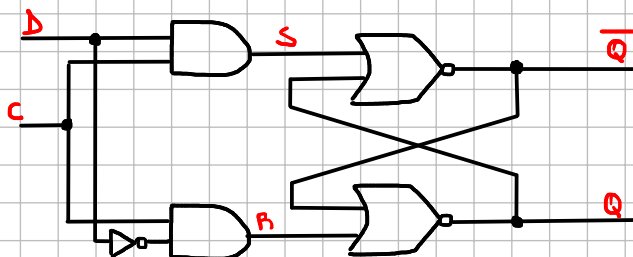
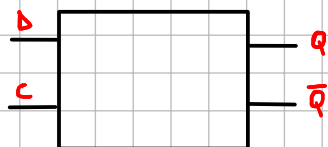
- $S=1, R=1$: STATO VIETATO (Q E $\bar{Q}$ POSSONO AVERE LO STESSO VALORE) → **RACE CONDITION**

IL PROBLEMA DI QUESTO CIRCUITO È PROPRIO LO **STATO INSTABILE / VIETATO** ED IL FATTO CHE SIA **ASINCRONO**.

PER RISOLVERE IL PROBLEMA DELLO STATO INSTABILE E L'ASINCRONIA È STATO IDEATO IL **LATCH-D**.

IL **LATCH-D** FUNZIONA ALLO STESSO MODO, HA PIÙ "SICURO", RICEVE COME INPUT **D** (IL DATO DA SCRIVERE) E IL **CLOCK**, CHE ABILITA O NO LA SCRITTURA DEL DATO.

IL SUO CIRCUITO :

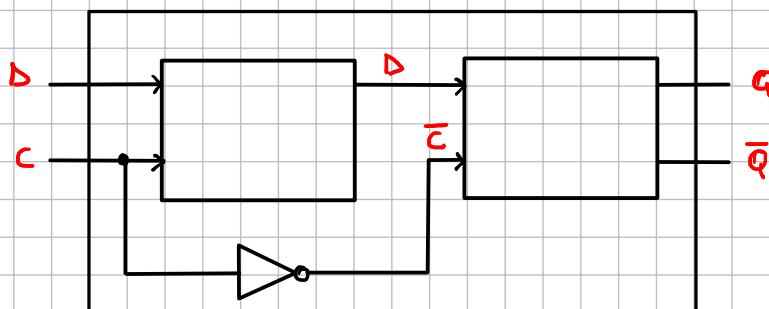


IN QUESTO MODO AVERE
SIA S CHE T = 1 È
IMPOSSIBILE, GRAZIE ALLE
AND INIZIALI E IL NOT.

IL PROBLEMA CHE PERSISTE È LA **TRASPARENZA** PERCHÉ QUANDO IL CLOCK È ALTO (1) CONTINUA A PERMETTERE DI SCRIVERE TUTTO IL TEMPO, OVVERO Q CAMBIA **ISTANTANEAMENTE** CON D, COME SE FOSSE UN CIRCUITO **APERTO**, QUESTO RENDE IMPOSSIBILE CREARE CIRCUITI PRECISI (COME **CONTATORI**) ED È PER QUESTO CHE BISOGNA IDEARE CIRCUITI SEQUENZIALI **EDGE TRIGGERED** (CHE SI ATTIVANO SUI FRONTI DEL CLOCK).

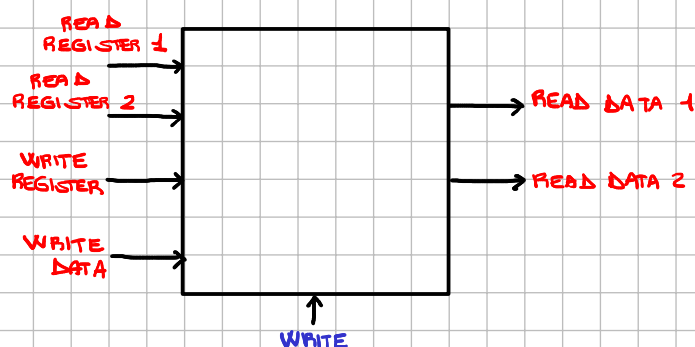
VENGONO COSÌ CREATI I **FLIP FLOP D** CHE RISOLVONO TUTTO :

I **FLIP FLOP D** SONO 2 LATCH-D MESSI IN SERIE, IN QUESTO MODO IL PRIMO LATCH-D "TRATTIENE IL DATO" D SENZA MODIFICARE DIRETTAMENTE Q, COSÌ IL SECONDO LATCH-D AL CAMBIAMENTO DEL CLOCK LASCIA MODIFICARE Q.

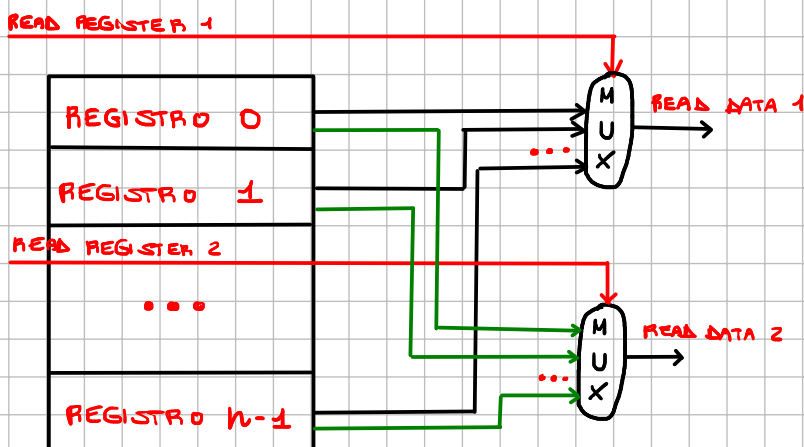


COA È UN REGISTER FILE ? (COME SCRIVE E COME LEGGE ?)

UN **REGISTER FILE** È UN INSIEME DI **REGISTRI** CHE POSSONO INVIARE DATI ALL'**ALU** E RICEVERE DATI DA **MEMORIZZARE**, IN INPUT RICEVE IL SEGNALE DI CONTROLLO **WRITE** CHE DECIDE SE BISOGNA SCRIVERE O LEGGERE DAI REGISTRI, SE **WRITE = 0** IN INPUT RICEVE I **2 REGISTRI DA LEGGERE**, SE **WRITE = 1** RICEVE IL **REGISTRO IN CUI SCRIVERE E IL DATO DA SCRIVERE**, E COME OUTPUT I **2 DATI LETTI**.



LEGGE UTILIZZANDO **2 MULTIPLEXER** COLLEGATI A TUTTI I REGISTRI, CHE RICEVONO COME SEGNALE DI CONTROLLO IL REGISTRO DA LEGGERE :



PER SCRIVERE SUI REGISTRI, SI UTILIZZA UN **DECODER**, DOVE L'INPUT BINARIO DI ESSO È IL **REGISTRO** IN CUI SCRIVERE CHE ABBIUTA ASSIEME AL **WRITE & CLOCK** UNA DELLE 2^n CELLE DI MEMORIA PER MEMORIZZARE IL **DATO DA SCRIVERE**.

